

Universidad del Rosario
Escuela de Ciencias e Ingeniería
Ingeniería de Datos

Parcial 2

Proyecto Integrador: Sistema NexoEventos

Cada grupo de 3 a 4 estudiantes construye un sistema de información completo para un **centro de convenciones**: una API REST en FastAPI que crea y gestiona la base de datos PostgreSQL con entidades de SQLAlchemy y DTO de Pydantic; triggers, funciones, consultas recursivas y funciones de ventana escritos directamente en PostgreSQL; y un frontend en Streamlit que consume la API.

1. El escenario de negocio

El Centro de Convenciones Nexo alquila salones y servicios a empresas y personas que organizan **congresos, conferencias, ferias, seminarios, eventos corporativos y sociales**. Hoy lo lleva todo en hojas de cálculo y necesita un sistema que **evite dobles reservas**, liquide cada evento automáticamente y mida cuánta gente llega de verdad.

Clientes. Quien contrata un evento es un cliente: una empresa (NIT) o una persona natural (cédula). El documento no se puede repetir. **Se registran nombre o razón social, correo, teléfono y fecha de registro**.

Salones. El centro tiene **seis salones de tres tamaños: pequeño, mediano y grande**. Cada salón tiene una capacidad máxima y un precio por hora. Un salón puede deshabilitarse temporalmente (**por mantenimiento**) sin borrarlo.

Eventos. Cada evento pertenece a un cliente y se realiza en un **solo salón, con fecha y hora de inicio y de fin**; puede terminar después de medianoche. Se registran **nombre, tipo, aforo esperado (las personas que contrata el cliente) y un coordinador del staff**. Un evento nace como **Cotizado**, pasa a **Confirmado** cuando el cliente acepta y termina como **Finalizado** o **Cancelado**; estos dos últimos estados son definitivos. Entre dos eventos del mismo salón debe quedar al **menos 1 hora libre** para montaje y aseo. Un evento cancelado libera el salón.

Servicios. Además del salón, el cliente contrata servicios del catálogo: **estaciones de agua y café, refrigerios (económico, medio o alto), sonido y multimedia**. Cada servicio se cobra por persona, por hora o por unidad. Al agregar un servicio a un evento se guardan la cantidad y el precio vigente en ese momento: si el centro sube sus tarifas después, los eventos ya cotizados no cambian. **El valor total del evento es el alquiler del salón (horas x precio por hora) más la suma de los servicios contratados**, y debe estar siempre actualizado.

Asistentes. Los visitantes se registran una sola vez (documento único, nombre, correo y empresa) y se inscriben a uno o varios eventos. Un evento no puede tener más inscritos que su aforo contratado. En la puerta se registra la **asistencia real** (check-in) con fecha y hora de llegada. Solo se permite desde una hora antes del inicio hasta la hora de fin, y una sola vez por persona. Al finalizar un evento, el centro quiere conocer su **tasa de asistencia real: personas que llegaron sobre personas inscritas.**

Staff. Cada empleado tiene nombre, cargo, área (Dirección, Operaciones, Comercial, Logística, Alimentos y Bebidas, Audiovisuales), correo, fecha de ingreso y **jefe inmediato**, que también es empleado del centro. La dirección general no tiene jefe. La organización tiene al menos cuatro niveles: **dirección, gerencias, jefaturas y coordinaciones, y auxiliares y técnicos.**

Trazabilidad. Todo cambio de estado, de salón o de horario de un evento queda registrado con el valor anterior, el valor nuevo, la fecha y el **usuario de la aplicación** que hizo el cambio.

Con base en esta descripción, usted debe definir el esquema de la base de datos: tablas, campos, tipos de datos y restricciones que considere necesarias.

2. Catálogo del centro y datos de prueba

Todos los grupos deben cargar exactamente estos salones y servicios, con estos precios. Así las cotizaciones son comparables entre grupos.

Salones

Salón	Tamaño	Capacidad (personas)	Precio por hora (COP)
Salón Orquídea	Pequeño	40	250.000
Salón Heliconia	Pequeño	60	320.000
Salón Guayacán	Mediano	150	650.000
Salón Ceiba	Mediano	200	800.000
Gran Salón Cóndor	Grande	600	1.800.000
Auditorio Principal	Grande	1.000	2.500.000

Servicios

Servicio	Categoría	Se cobra	Precio unitario (COP)
Estación de agua	Bebidas	Por persona	3.000
Estación de café	Bebidas	Por persona	6.500
Refrigerio económico	Refrigerios	Por persona	12.000
Refrigerio medio	Refrigerios	Por persona	22.000
Refrigerio alto	Refrigerios	Por persona	38.000
Sonido básico (2 micrófonos + parlantes)	Audiovisuales	Por hora	120.000
Sonido profesional (consola + 6 micrófonos)	Audiovisuales	Por hora	350.000
Multimedia (proyector + pantalla)	Audiovisuales	Por hora	90.000
Streaming y grabación	Audiovisuales	Por hora	450.000

Datos de prueba mínimos. El resto de los datos lo genera cada grupo, en un script SQL o en un script de Python, cargado después de crear las tablas y los triggers:

- Al menos 8 clientes (empresas y personas).
- Al menos 15 personas de staff en 4 niveles de jerarquía.
- Al menos 12 eventos en todos los estados, pasados y futuros, con servicios configurados.
- Al menos 60 asistentes e inscripciones suficientes para demostrar cupos llenos, check-ins y tasas de asistencia distintas.

Los datos deben pasar por los triggers (no se desactivan para cargarlos): valores totales, tasas de asistencia y auditoría quedan calculados por la propia base.

3. Módulo A: lógica procedimental (triggers y funciones)

Las tablas las crean las entidades de SQLAlchemy. Todo lo que el ORM no puede expresar se escribe directamente en PostgreSQL, en uno o varios scripts .sql que se aplican después de crear las tablas. Esas reglas deben vivir en la base: si alguien inserta datos desde psql o pgAdmin, sin pasar por la API, se tienen que cumplir igual. Validar también en la API o en Streamlit es bienvenido, pero no reemplaza el trigger.

1. **A1. Disponibilidad y capacidad del salón (trigger).** Bloquea crear o modificar un evento si el aforo esperado supera la capacidad del salón, si el salón está deshabilitado o si se cruza con otro evento no cancelado del mismo salón, contando la hora de montaje entre eventos. Debe actuar también al reprogramar (cambio de salón, horario o aforo) y al reactivar un evento. **El mensaje de error debe decir con qué evento y en qué horario se cruza.**
2. **A2. Cupos y asistencia real (trigger).** No se permiten inscripciones en eventos cancelados o finalizados, ni por encima del aforo contratado. **El check-in** solo se acepta desde una hora antes del inicio hasta la hora de fin del evento, y una sola vez por asistente.
3. **A3. Liquidación automática del evento (triggers).** El valor total del evento se mantiene correcto sin que la aplicación lo calcule: al crear el evento, al agregar, modificar o quitar servicios y al reprogramarlo. Al contratar un servicio se congela su precio de catálogo. Si no se indica la cantidad, se usa el aforo esperado para los servicios por persona y la duración del evento (en horas, redondeada hacia arriba) para los servicios por hora. No se pueden modificar los servicios de un evento finalizado o cancelado.
4. **A4. Cierre del evento y auditoría (triggers).** *Finalizado y Cancelado* son estados definitivos. Un evento solo se puede finalizar cuando su hora de fin ya pasó; al hacerlo se calcula y guarda la tasa de asistencia real (porcentaje con dos decimales, vacía si no hubo inscritos). Cada cambio de estado, salón u horario queda en una tabla de auditoría con valor anterior, valor nuevo, fecha y usuario de la aplicación.
5. **A5. Salones disponibles (función).** **Recibe** inicio, fin y aforo, y devuelve los salones activos con capacidad suficiente y sin cruces (incluida la hora de montaje), con el costo del salón para ese horario, del más pequeño que sirva al más grande. La aplicación la usa para verificar disponibilidad y para que al crear un evento solo se ofrezcan salones libres.

Pista para A4. La API se conecta a la base siempre con el mismo usuario, así que `current_user` no sirve para saber quién hizo el cambio. Investigue cómo pasar un valor a la base durante una transacción con `set_config(...)` y leerlo desde el trigger con `current_setting(...)`.

Pista para la integración con el ORM. Después de `db.commit()`, los campos que calculó un trigger no están en el objeto de Python hasta que se hace `db.refresh(...)`. Y si un trigger lanza `RAISE EXCEPTION`, SQLAlchemy lo entrega como una excepción cuyo error original de PostgreSQL está en `exc.orig`.

4. Módulo B: consultas de inteligencia de negocio (SQL avanzado)

Cree funciones o vistas en PostgreSQL que resuelvan estos cuatro reportes. La API los consulta con SQL parametrizado (por ejemplo, `text(...)` de SQLAlchemy); no se reescriben con el ORM.

1. **B1. Organigrama (CTE recursiva).** Dado un miembro del staff, o toda la organización si no se indica ninguno, lista a todas las personas que dependen de él directa o indirectamente, con su nivel, su ruta de mando («Directora > Gerente > Jefe») y el total de personas que tiene a cargo, directas e indirectas.
2. **B2. Cadena de escalamiento (CTE recursiva).** Dado un miembro del staff, lista en orden a sus jefes hasta la dirección general. Responde a la pregunta «a quién escalo un incidente durante un evento».
3. **B3. Ranking de clientes y análisis de Pareto (funciones de ventana).** Por cliente: número de eventos no cancelados, total facturado, posición en el ranking, porcentaje de participación sobre el total y porcentaje acumulado.
4. **B4. Ocupación de salones y tiempos muertos (funciones de ventana).** Por salón y en orden cronológico, cada evento no cancelado con su número dentro del salón, el evento anterior, los días que el salón estuvo libre desde que terminó el anterior y las horas reservadas acumuladas.

Pistas. Para B3, compare `SUM(...) OVER ()` con `SUM(...) OVER (ORDER BY ...)`. Para B4, use `LAG(...)` con `PARTITION BY`. En B1, proteja la recursión contra ciclos en la jerarquía.

5. Desarrollo del sistema: API en FastAPI y frontend en Streamlit

El sistema tiene tres capas. Streamlit nunca se conecta directamente a la base de datos: toda operación pasa por la API.

5.1 Backend (FastAPI + SQLAlchemy + Pydantic)

Siga la estructura del proyecto de ejemplo visto en clase:

- `main.py`: crea las tablas con `Base.metadata.create_all(...)`, registra los routers y el manejo de errores.
- `app/database.py`: `engine`, `SessionLocal`, `Base` y la dependencia `get_db`.
- `app/models.py`: una entidad (`XxxEntity`) por tabla, con llaves primarias y foráneas, `unique`, `nullable`, `CheckConstraint`, relaciones y la auto-referencia del staff.
- `app/schemas.py`: DTO de Pydantic separados de las entidades: `XxxCreate` para validar lo que entra y `XxxResponse` (con `from_attributes`) para lo que sale.
- `app/routers/`: un router por recurso.
- `sql/`: los scripts de triggers, funciones y vistas. Deben poder ejecutarse después de `create_all` sin errores; se recomienda que sean idempotentes (`CREATE OR REPLACE`) para aplicarlos al iniciar la API.

Requisitos adicionales:

- Códigos HTTP coherentes: 201 al crear, 404 si no existe, 409 cuando un trigger rechaza la operación, 422 cuando los datos son inválidos. La respuesta 409 debe incluir el mensaje de la regla violada, no una traza de Python.
- Usuario de la aplicación: la API recibe quién hace cada operación (por ejemplo, en un encabezado HTTP X-Usuario) y lo entrega a la base para la auditoría de A4.
- Seguridad: SQL siempre parametrizado; la cadena de conexión en una variable de entorno o archivo .env, no escrita en el código.
- Documentación: los endpoints se ven descritos en /docs (Swagger).

Endpoints mínimos. Los nombres de las rutas pueden variar; la funcionalidad no.

Método	Ruta sugerida	Qué hace
GET, POST	/clientes/	Listar o buscar y registrar clientes
GET	/clientes/{id}/eventos	Eventos de un cliente
GET, POST	/staff/	Listar y registrar staff con su jefe inmediato
GET	/salones, /servicios	Catálogo
GET	/salones/disponibles?inicio=&fin=&aforo=	Salones disponibles (usa A5)
GET, POST	/eventos/	Listar con filtros y crear eventos
GET	/eventos/{id}	Detalle con servicios y resumen de asistencia
PATCH	/eventos/{id}	Reprogramar: salón, horario, aforo
PATCH	/eventos/{id}/estado	Confirmar, finalizar o cancelar
POST, PUT, DELETE	/eventos/{id}/servicios	Configurar los servicios del evento
GET	/eventos/{id}/cotizacion	Desglose de costos: salón + cada servicio = total
GET	/eventos/{id}/auditoria	Historial de cambios del

Método	Ruta sugerida	Qué hace
		evento
GET, POST	/asistentes/	Listar o buscar y registrar asistentes
GET, POST	/eventos/{id}/inscripciones	Inscritos del evento e inscribir
POST	/eventos/{id}/checkin	Registrar asistencia real por documento
GET	/reportes/...	Un endpoint por reporte B1 a B4

5.2 Frontend (Streamlit)

Módulo	Debe permitir
Inicio	Tablero con eventos por estado, próximos eventos y tasa de asistencia promedio
Eventos	Crear un evento eligiendo solo entre salones disponibles; configurar servicios; ver la cotización; confirmar, finalizar o cancelar; reprogramar; ver la auditoría
Asistentes y check-in	Registrar asistentes, inscribirlos y hacer check-in en puerta por documento; ver inscritos y quiénes llegaron
Disponibilidad de salones	Consultar por fecha, horario y número de personas
Cientes y staff	Registrar, buscar y ver los eventos de cada cliente; registrar staff con su jefe
Reportes	B1 a B4, al menos uno con gráfico

Cuando la API responde 409 o 422, Streamlit muestra el mensaje de la regla de negocio de forma clara y sigue funcionando. Si la API no está disponible, lo informa en lugar de mostrar una traza.

6. Requerimientos de diseño

- **Normalización:** el esquema que generan las entidades está al menos en tercera forma normal.
- **Tipos de datos:** los estudiantes deducen los tipos adecuados: fechas con hora para inicio, fin y check-in; valores numéricos exactos para dinero (no Float); textos con longitud; booleanos.
- **Restricciones en las entidades:** llaves primarias y foráneas, unique, nullable y CheckConstraint para los valores permitidos (tipos, estados, tamaños, áreas, capacidad positiva, fin posterior al inicio).
- **Reglas en la base:** A1 a A5 y B1 a B4 viven en PostgreSQL, no solo en Python.
- **Gestión de errores:** ni la API ni Streamlit se “rompen” si un trigger lanza una excepción; el usuario ve qué regla de negocio se incumplió. Se puede mostrar información traída directamente de la base.
- **Reproducibilidad:** el repositorio incluye requirements.txt para API y frontend, y un README con los pasos para levantar todo desde una base vacía.

7. Entregables

Entregable 1 – Backend: Octubre 9, antes de iniciar la clase (50% de la nota)

- **Código** (apellido1_apellido2_....zip) con las carpetas de la API (incluida sql/) y el README.
- **Scripts SQL** de triggers, funciones y vistas, y el script o programa de carga de datos de prueba.
- **Reporte con evidencias:** para cada regla A1 a A5, un caso permitido y uno bloqueado, con la petición a la API (Swagger o similar) y la pantalla de Streamlit. Para B1 a B4, la comparación entre el resultado calculado a mano y el de la plataforma, presentada y explicada.
- **Documentación:** diagrama relacional, diccionario de datos, lista de endpoints con un ejemplo de petición y respuesta.

Comentado [VB1]: Librería para el front

Comentado [VB2]: Descripción de las tablas

Entregable 2 – Frontend: Octubre 19, antes de iniciar la clase (35% de la nota)

- **Código completo** (apellido1_apellido2_....zip) con las carpetas de la API (incluida sql/), el front y el README.
- **Documentación:** Manual de rutas, y manual de usuario del frontend.

Comentado [VB3]: Instrucciones

Presentación en plenaria de 15 minutos: Octubre 19 y 21. (15% de la nota).

Debe incluir:

- o Esquema de la base de datos sustentado (entidades y restricciones).
- o Diseño de los triggers y cómo la API traduce sus errores.
- o Demostración en vivo desde Streamlit de al menos dos reglas bloqueando una operación y de los reportes.

- El trabajo se desarrolla en **grupos de 3 a 4 personas**.

Uso de IA generativa. La IA generativa debe estar restringida a las siguientes actividades:

- Explicación de temas generales del curso o de códigos provistos en las presentaciones. **No se puede usar para generar código del examen.**
- Generación de datos iniciales, **dado un esquema creado por ustedes.**
- Corrección de estilo del reporte.

En cualquier caso en que se use IA generativa, debe entregarse el enlace a la conversación completa.

8. Rúbrica de evaluación

Puntaje total: 5 puntos · **Duración:** 4 semanas · **Equipo:** 3 a 4 personas

8.1 Modelado y estructura de datos (15 %)

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
Normalización	El esquema está en 3FN, con relaciones coherentes y sin redundancia innecesaria.	2FN o errores menores de redundancia.	No está normalizado o faltan llaves primarias o foráneas.
Entidades y restricciones	Todas las entidades de la narrativa (clientes, eventos, salones, servicios, servicios por evento, asistentes, inscripciones, staff con auto-referencia, auditoría) como XxxEntity, con tipos adecuados y CheckConstraint, unique y nullable correctos.	Faltan 1 o 2 entidades o atributos clave, o los tipos no son los adecuados (p. ej. Float para dinero).	Faltan entidades críticas para el funcionamiento del sistema.
Datos de prueba	Cumplen los mínimos, pasan por los triggers y	Incompletos o no cubren todos los	No hay datos o no permiten probar las

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
	permiten demostrar cada regla.	estados.	reglas.

8.2 Lógica programada: triggers y funciones (25 %)

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
A1 Disponibilidad y capacidad	Bloquea cruces con la hora de montaje, aforo mayor a la capacidad y salones deshabilitados, también al reprogramar. El mensaje indica el evento en conflicto.	Funciona al crear pero no al reprogramar, o ignora la hora de montaje.	No implementa el trigger o no bloquea la transacción.
A2 Cupos y check-in	Controla cupo, estado del evento, ventana horaria y doble check-in.	Falta una de las condiciones.	No implementa el trigger.
A3 Liquidación automática	El valor total es correcto al crear, al cambiar servicios y al reprogramar; congela el precio y calcula la cantidad por defecto.	Calcula el valor pero no lo actualiza en algún caso, o no congela el precio.	No hay cálculo automático en la base.
A4 Cierre y auditoría	Estados definitivos, tasa de asistencia correcta y auditoría con el usuario de la aplicación (no solo el de la base).	La auditoría no guarda el usuario de la aplicación o la tasa es incorrecta.	No existe la tabla o el trigger de auditoría.
A5 Salones disponibles	Función correcta, ordenada y con costo, usada por la aplicación.	Resultados con errores (p. ej. ignora la hora de montaje) o no se usa en la aplicación.	No existe la función.

8.3 Consultas analíticas avanzadas (20 %)

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
B1 Organigrama	CTE recursiva con todos los niveles, ruta de mando y personas a cargo.	Solo un nivel de profundidad o sin el conteo de personas a cargo.	No utiliza CTE recursiva.
B2 Escalamiento	Cadena completa y ordenada hasta la dirección general.	Cadena incompleta.	No implementa la consulta.
B3 Ranking y Pareto	Ranking, participación, acumulado y segmento del 80 % correctos con funciones de ventana.	Usa funciones de ventana pero el acumulado o el segmento es incorrecto.	No utiliza funciones de ventana.
B4 Ocupación de salones	LAG, particiones y orden correctos; días libres y horas acumuladas bien calculados.	Errores de partición u orden.	No utiliza funciones de ventana.

8.4 Backend en FastAPI (15 %)

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
Arquitectura y endpoints	Sigue la estructura de clase (entidades, DTO, routers); cubre todos los endpoints mínimos y funcionan.	Faltan algunos endpoints o mezcla entidades y DTO.	La API no funciona o no se conecta a la base.
Errores y validación	DTO con validaciones; códigos 201, 404, 409 y 422 coherentes; el 409 trae el mensaje del trigger.	Valida, pero algunos errores de la base llegan como 500 o con mensajes técnicos.	Sin validación ni manejo de errores.

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
Buenas prácticas	SQL parametrizado, refresh tras los triggers, credenciales fuera del código y Swagger documentado.	Cumple parcialmente.	Concatena SQL o tiene credenciales en el código.

8.5 Frontend en Streamlit (10 %)

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
Funcionalidad	Todos los módulos funcionan y consumen la API.	Algunos módulos faltan o no están conectados.	No funciona o se conecta directamente a la base.
Errores y usabilidad	Muestra las reglas violadas con claridad y nunca se cierra; formularios claros.	En algunos casos muestra trazas técnicas o se detiene.	Errores ilegibles o la aplicación se cae.

8.6 Funcionalidad adicional (5 %)

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
Funcionalidad del numeral 5.3	Implementada e integrada en base, API y frontend.	Implementada parcialmente.	No implementada.

8.7 Documentación y presentación (10 %)

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
Documentación	Diagrama relacional, diccionario de datos, endpoints, manual de usuario y README reproducible; código comentado.	Entrega el diagrama pero no explica los triggers, o el README no permite levantar el sistema.	No hay documentación.
Presentación	Sustenta el diseño y hace la demostración en vivo de reglas y	Demostración parcial.	Sin demostración.

Criterio	Excelente (100 %)	Aceptable (70 %)	Insuficiente (0–40 %)
	reportes.		

9. Ejemplos de comportamiento esperado

Estos ejemplos son ilustrativos: salen de una implementación de referencia con sus propios datos de prueba. No es obligatorio usar los mismos nombres de campos ni la misma estructura del JSON, pero sí el mismo comportamiento.

Consultar salones disponibles (A5). El 18 de noviembre de 9:00 a 11:00 para 50 personas no se ofrecen el Salón Orquídea (capacidad 40) ni los salones ocupados. El primero es el más ajustado al aforo.

```
GET /salones/disponibles?inicio=2026-11-18T09:00&fin=2026-11-18T11:00&aforo=50
```

```
[
  {"salon_id": 2, "nombre": "Salón Heliconia", "tamano": "Pequeño", "capacidad": 60,
  "precio_hora": 320000.0, "costo_estimado": 640000.0},
  {"salon_id": 4, "nombre": "Salón Ceiba", "tamano": "Mediano", "capacidad": 200,
  "precio_hora": 800000.0, "costo_estimado": 1600000.0},
  {"salon_id": 5, "nombre": "Gran Salón Cóndor", "tamano": "Grande", "capacidad": 600,
  "precio_hora": 1800000.0, "costo_estimado": 3600000.0}
]
```

Crear un evento que se cruza con otro (A1). El Salón Guayacán tiene un evento hasta las 12:00; uno que empieza a las 12:30 no deja la hora de montaje.

```
POST /eventos/
{"cliente_id": 2, "salon_id": 3, "nombre": "Taller de Economía Circular", "tipo":
"Seminario",
"inicio": "2026-11-18T12:30", "fin": "2026-11-18T14:00", "aforo_esperado": 50}
```

```
HTTP 409
{"detail": "DISPONIBILIDAD: El salón \"Salón Guayacán\" está reservado para \"Jornada
Científica Solvida\" (2026-11-18 08:00 a 2026-11-18 12:00). Se requiere 1 hora de montaje
entre eventos.",
"codigo_sql": "P0001", "regla": "DISPONIBILIDAD", "sugerencia": "Consulte los salones
disponibles para ese horario."}
```

Inscribir por encima del aforo y check-in fuera de horario (A2).

```
HTTP 409
{"detail": "CUPO: El evento \"Club de Lectura Páramo\" alcanzó su aforo contratado (20
personas).", "regla": "CUPO"}
```

HTTP 409

```
{"detail": "CHECK-IN: Solo se permite ingresar entre 2026-11-18 06:30 y 2026-11-18 18:30 (hora indicada: 2026-11-18 05:00).", "regla": "CHECK-IN"}
```

Finalizar un evento (A4). La base calcula la tasa de asistencia (32 de 45 inscritos llegaron) y la auditoría guarda el usuario que envió la aplicación, no el de la base de datos.

```
PATCH /eventos/8/estado      X-Usuario: farango
{"estado": "Finalizado"}
```

HTTP 200

```
{"id": 8, "nombre": "Capacitación en Ciberseguridad", "estado": "Finalizado",
"valor_total": 2565000.0, "tasa_asistencia": 71.11}
```

GET /eventos/8/auditoria (última fila)

```
{"campo": "estado", "valor_anterior": "Confirmado", "valor_nuevo": "Finalizado",
"usuario": "farango", "fecha": "2026-09-24T22:55:30"}
```

Datos inválidos detectados por el DTO, antes de llegar a la base. Un evento con la hora de fin anterior al inicio responde HTTP 422 con el mensaje «La fecha/hora de fin debe ser posterior a la de inicio».